

Algorithms for massive data

Market-Basket Analysis Project

Matteo Hasa — 56269A

MSc in Data Science for Economics

Università degli Studi di Milano

July 6, 2026

Contents

1	Introduction	3
2	Data Exploration and Preprocessing	3
3	Algorithm Implementation	4
4	Scalability Proof	5
5	Experiments	6
6	Discussion	7

"I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study."

1 Introduction

The goal of this project is to perform a Market-Basket Analysis, which consists in finding the itemsets that occur frequently in a collection of baskets. In this case every movie is a basket and the actors playing in it are the items, so a frequent itemset is a group of actors who often appear in the same films. On top of the frequent itemsets were also extracted association rules, which says how strongly the presence of some actors implies the presence of others.

Three algorithms have been implemented from scratch and are described in the following sections: **Apriori**, **PCY** and **SON**. Differently from a pairs-only version, the Apriori implemented here returns frequent itemsets of any size, so the analysis also finds frequent triples of actors. The SON algorithm is built on top of Apache Spark, so the same code could scale to datasets that do not fit into the memory of a single machine.

2 Data Exploration and Preprocessing

The dataset used is the IMDB Movies Dataset¹, which contains the top 1000 movies and TV shows of the IMDB website. It is downloaded directly inside the notebook through the Kaggle API.

Besides title, genre, runtime and rating, each row reports the four leading actors of the movie in the columns *Star1*, *Star2*, *Star3* and *Star4*, which are the only columns relevant to this analysis. The CSV file is loaded into a Pandas DataFrame and only the four actor columns are kept. From them a list called *baskets* is built: each basket is the set of the actors of one movie. During this step the names are cleaned (extra spaces and missing values are removed) and the movies left with fewer than two actors are discarded, since they cannot produce any co-occurrence. This leaves 1000 usable baskets.

A global variable `USE_FULL_DATASET` allows switching between the whole dataset and a smaller reproducible sample: this is useful to test the algorithms quickly and shows that the code works the same way on a subset of the data, as one would do on a truly massive dataset. The support threshold is expressed as a fraction of the number of baskets, so the configuration keeps its meaning if the data size changes.

¹<https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows>

3 Algorithm Implementation

The simplest algorithm is **Apriori**, which finds the frequent itemsets with a small number of passes over the data. It is based on the monotonicity property: every subset of a frequent itemset is also frequent, so a k -itemset can be frequent only if all its $(k - 1)$ -subsets are. The implementation works level by level:

1. **Single items.** The data are scanned once and every actor is counted. Only the actors whose count reaches the support threshold s are kept (the frequent items).
2. **Candidate generation.** For each level k , candidate k -itemsets are built by joining the frequent $(k - 1)$ -itemsets, and any candidate having a non-frequent subset is immediately discarded (pruning).
3. **Counting.** The surviving candidates are counted against the baskets and those above s become the new frequent itemsets. The procedure repeats until no new frequent itemset is found (here at most size four, since a basket has four actors).

Apriori may waste a lot of memory when counting pairs, which is the reason for its variants. The **PCY** algorithm (Park, Chen and Yu) uses the memory that is free during the first pass:

1. **First pass.** While the single items are counted, every pair of a basket is also hashed into one of the buckets of a hash table, and the bucket counter is increased.
2. **Between the passes.** The bucket counters are replaced by a much smaller *bitmap*: a bucket is marked as frequent only if its counter reaches s . A pair that falls into an infrequent bucket cannot be frequent and can be ignored.
3. **Second pass.** A pair is counted only if both its items are frequent and it hashes into a frequent bucket, so many candidate pairs are removed before counting.

The last algorithm is **SON** (Savasere, Omiecinski and Navathe). Its advantage is that it can be parallelized, which is why it was implemented

on Apache Spark, the framework designed to run MapReduce jobs and scale horizontally. Even if our data are not massive, the implementation uses exactly the operations that would scale to a real cluster. It works in two passes:

1. **Map pass.** The baskets are split into chunks and turned into RDDs. On each chunk the Apriori algorithm is run with a local threshold proportional to the chunk size; the union of all the locally frequent itemsets is the set of candidates. Because a globally frequent itemset must be frequent in at least one chunk, this step never loses a true frequent itemset.
2. **Reduce pass.** The candidates are broadcast and counted over the whole dataset with a `flatMap/reduceByKey` job, keeping only those that reach the global threshold s . This removes the false positives produced locally.

Finally, from the frequent itemsets the **association rules** are generated. For a frequent itemset split into an antecedent A and a consequent C , the rule $A \Rightarrow C$ is described by its confidence $\text{supp}(A \cup C)/\text{supp}(A)$ and by its lift, i.e. the confidence divided by $\text{supp}(C)/N$ (with N the number of baskets). A lift above one means that A and C appear together more often than they would by chance. All the supports needed are already available among the frequent itemsets, so no extra scan of the data is required.

Itemsets are stored in dictionaries whose keys are the itemsets themselves (kept as `frozensets`, so that Spark can use them as keys) and whose values are the counts. This is convenient because the data are sparse: an actor who appears in few films does not need to be paired with all the others, which would waste space.

4 Scalability Proof

To assess scalability the three algorithms were run on 30%, 60% and 100% of the data and their execution times were compared. The results are shown in Table 1.

PCY is by far the fastest (about 0.007 s on the full data). Since each basket has only four actors, PCY hashes very few pairs and, after the bitmap step, counts only the pairs that survive; this avoids the heavy work that the

Fraction of Data	Apriori (s)	PCY (s)	SON (s)
30%	52.1541	0.0054	6.6284
60%	7.9485	0.0047	25.8712
100%	9.4826	0.0073	100.2357

Table 1: Execution times of Apriori, PCY and SON on different fractions of the data.

general Apriori does. **Apriori** is much slower (about 9 s): because it looks for itemsets of every size, it has many frequent single actors and builds a large number of candidate pairs from them, all of which must be counted. **SON** has the highest times and, differently from the other two, its time grows clearly with the data (6.6 $\rightarrow$ 25.9 $\rightarrow$ 100.2 s): on a dataset this small the cost is dominated by Spark’s fixed overhead (scheduling, serialization and the shuffle of the candidates). The important point is that this overhead is exactly the price to pay for distributing the computation: Apriori and PCY are limited by the memory of one machine, while SON is the only algorithm that could keep running on data that exceeds it. The three algorithms were also checked to return the same frequent itemsets on every fraction.

5 Experiments

For all the algorithms the support threshold was set to 3 (i.e. `MIN_SUPPORT` = 0.003 on 1000 movies): an itemset is frequent if it appears in at least three films. A higher threshold would be too strict and would leave almost nothing, while a lower one would fill the result with noise. For PCY the number of buckets was set to 5000, which is large enough to keep the hash table effective without wasting memory. For SON the data were split into 4 chunks; the local threshold is therefore `MIN_SUPPORT` $\times$ |chunk|, so on small chunks the local Apriori is permissive and mainly acts as a candidate generator, while the real filtering happens in the global reduce pass.

With these parameters the analysis finds **299 frequent itemsets**. Most of them are single, very prolific actors; the interesting part is the multi-actor itemsets, shown in Table 2, which include three frequent *triples*.

From the frequent itemsets, **46 association rules** with confidence at least 0.5 were generated. The strongest ones are reported in Table 3. As expected for trilogy casts, the confidence is often 1 and the lift reaches several

Size	Actors	Support
3	Daniel Radcliffe, Emma Watson, Rupert Grint	5
3	Carrie Fisher, Harrison Ford, Mark Hamill	3
3	Elijah Wood, Ian McKellen, Orlando Bloom	3
2	Daniel Radcliffe, Rupert Grint	6
2	Daniel Radcliffe, Emma Watson	5
2	Emma Watson, Rupert Grint	5
2	Tim Allen, Tom Hanks	4
2	Joe Pesci, Robert De Niro	4
2	Harrison Ford, Mark Hamill	3
2	Chris Evans, Robert Downey Jr.	3
2	Mark Ruffalo, Robert Downey Jr.	3
2	Al Pacino, Robert De Niro	3
2	Christian Bale, Michael Caine	3
2	Ethan Hawke, Julie Delpy	3

Table 2: Some of the frequent itemsets found (support is the number of films).

hundreds, meaning that those actors co-occur far more than they would by chance.

Finally, as a correctness check the notebook verifies that SON returns exactly the same itemsets as the in-memory a-priori and that PCY returns the same pairs as a-priori; the assertion passes, so the three implementations agree.

6 Discussion

The frequent itemsets can be grouped in two intuitive families. The first one is made of *franchise* groups: most of the pairs, and all three triples, come from film series whose cast is the same in every chapter, such as Harry Potter (Radcliffe, Watson, Grint), Star Wars (Hamill, Ford, Fisher), The Lord of the Rings (Wood, McKellen, Bloom) and the Marvel movies (Downey Jr., Evans, Ruffalo). The second family is made of *famous duos* who worked together many times, like Al Pacino & Robert De Niro, Robert De Niro & Joe Pesci or Tom Hanks & Tim Allen. The association rules confirm this picture: within a trilogy each actor implies the others with very high confidence and lift.

Antecedent	Consequent	Conf.	Lift
Mark Hamill	Carrie Fisher, Harrison Ford	1.00	333.3
Carrie Fisher, Harrison Ford	Mark Hamill	1.00	333.3
Elijah Wood	Ian McKellen, Orlando Bloom	1.00	333.3
Harrison Ford, Mark Hamill	Carrie Fisher	1.00	250.0
Carrie Fisher	Harrison Ford, Mark Hamill	0.75	250.0
Daniel Radcliffe	Rupert Grint	1.00	166.7
Emma Watson, Rupert Grint	Daniel Radcliffe	1.00	166.7
Daniel Radcliffe	Emma Watson, Rupert Grint	0.83	166.7
Ethan Coen	John Turturro	0.50	166.7

Table 3: Strongest association rules (confidence ≥ 0.5), ranked by lift.

About the algorithms, the experiments show two different ideas of efficiency. PCY is the fastest in memory, because its hash-and-bitmap trick removes the many useless candidate pairs that the general a-priori still counts. SON is the slowest here and its time grows with the data, but it is the only one that distributes the work and would therefore keep running on a dataset too large for a single machine; on our small dataset its parallelization is pure overhead. In other words, the advantages of PCY and SON are not fully visible on a non-massive dataset, but the implementation shows the techniques that would make the difference at scale. A natural limit of this analysis is that each movie lists only four actors, so no itemset larger than four can ever be found; the same code would apply unchanged to richer cast lists or a much larger catalogue.

References

- [1] Anand Rajaraman and Jeffrey David Ullman. *Mining of Massive Datasets*. Cambridge University Press, 2011. Available at <http://www.mmds.org/>.